

BÁO CÁO GIỮA KỲ

KHÓA LUẬN TỐT NGHIỆP

**Thiết kế CNN Accelerator cho phân loại rối loạn nhịp tim ECG trên
FPGA Intel Cyclone V**

Sinh viên thực hiện: Lê Minh Đức

Giảng viên hướng dẫn : ThS. Nguyễn Thị Thiên Trang
: ThS. Mã Khải Minh

Ngày báo cáo: 01/07/2026

1. Tóm tắt

Khóa luận thiết kế và triển khai một lõi IP CNN Accelerator nhẹ (lightweight) trên FPGA Intel Cyclone V nhằm phân loại rối loạn nhịp tim từ tín hiệu điện tim (ECG) đơn đạo trình (lead II). Dự án gồm hai phần gắn kết chặt chẽ: (i) một pipeline phần mềm PyTorch thực hiện huấn luyện — cắt tỉa (pruning) — lượng tử hóa INT8 lũy thừa hai (power-of-two), và (ii) một lõi phần cứng Verilog đã được kiểm chứng bit-exact với mô hình phần mềm và chạy thực tế trên kit DE10-Standard.

Các kết quả chính tính đến thời điểm báo cáo:

- **Độ chính xác:** $\sim 94\%$ / macro-F1 $\sim 0,93$ trên tập Chapman (4 lớp AFIB/GSVT/SB/SR, chia patient-independent 70/15/15), với mô hình chỉ 654 tham số.
- **Bit-exact:** 21 điểm kiểm tra (checkpoint) mỗi mẫu khớp tuyệt đối giữa Python và RTL — $\max|\text{diff}| = 0$ LSB trên 15.312 phần tử so sánh. Lượng tử hóa INT8 không gây mất mát so với float32 (cùng $\sim 94\%$).
- **Phần cứng:** một inference hoàn tất trong 5.216 chu kỳ $\approx 52,16 \mu\text{s}$ @ 100 MHz (xác định, không biến thiên), dùng 2.201 ALM (5%) và 28 DSP (25%) của thiết bị 5CSXFC6D6F31C6; $F_{\text{max}} = 104,85 \text{ MHz}$.
- **Demo trên board:** DE10-Standard qua cầu JTAG-to-Avalon đạt 94,27% (1004/1065), tải lập kết quả mô phỏng.
- **Cross-dataset:** mô hình huấn luyện trên bộ Chapman mở rộng (Chapman/Ningbo) đánh giá zero-shot trên Georgia (PhysioNet 2020, 12-lead, 5552 record) đạt 93,07% / macro-F1 0,9149 / macro-AUC 0,9812, chứng minh lõi tổng quát hóa thật ra ngoài dữ liệu huấn luyện.

2. Mục tiêu đề tài và bài toán

Mục tiêu thiết kế cụ thể của đề tài:

1. Thiết kế một mô hình CNN 1D nhỏ gọn, đạt độ chính xác $\geq \sim 94\%$ trên 4 lớp nhịp Chapman, trên cơ sở tham khảo và tối ưu kiến trúc của bài báo Liu (2023).
2. Lượng tử hóa INT8 với khâu rescale không cần multiplier (power-of-two) nhằm giảm tài nguyên DSP và năng lượng tiêu thụ.
3. Triển khai lõi RTL được kiểm chứng bit-exact với mô hình phần mềm (không phải chỉ là “xấp xỉ INT8” như đa số công trình trước).
4. Kiểm chứng chức năng thiết kế bằng mô phỏng và triển khai, đo đạc thực tế trên board DE10-Standard.

- Kiểm tra khả năng tổng quát hóa (cross-dataset) trên dữ liệu Georgia 12-lead để đánh giá độ bền của mô hình ngoài phân phối huấn luyện.

3. Đề xuất mô hình và tối ưu pipeline (Software)

3.1. Kiến trúc mô hình đề xuất

Mô hình ECG_1DCNN (bản đã pruned) gồm 4 lớp tích chập một chiều (kênh 4-4-8-8, kernel $K=5$, padding=2), mỗi lớp theo sau bởi MaxPool stride 5; tiếp đến là Global Average Pooling (GAP) $\rightarrow$ lớp Fully-Connected ($8 \rightarrow 4$) $\rightarrow$ argmax. Kiến trúc kế thừa tinh thần “fully-mapped 1D-CNN” nhưng được thu nhỏ mạnh (số kênh là lũy thừa 2 để thuận lợi cho phần cứng) và cắt tỉa kênh để giảm tham số xuống còn 654 — cực nhỏ, hướng tới thiết bị đeo.

Bảng 1. Cấu trúc mô hình ECG_1DCNN pruned (654 tham số).

Lớp	In_ch	Out_ch	K	Pool	ReLU	In_len	Out_len
Conv1	1	4	5	/5	Không	2500	500
Conv2	4	4	5	/5	Không	500	100
Conv3	4	8	5	/5	Không	100	20
Conv4	8	8	5	/5	Có	20	4
GAP	8	8	—	/4	—	4	1
FC	8	4	—	—	—	1	1

- ReLU chỉ đặt sau Conv4 — nhằm giữ lại đặc trưng âm của tín hiệu ECG ở Conv1–Conv3.
- Đầu vào: 2500 mẫu INT8 (5 giây @ 500 Hz, lead II). Đầu ra: 4 lớp AFIB / GSVT / SB / SR.

3.2. Thuật toán suy luận của mô hình CNN

Lan truyền tiến (forward) của mô hình là chuỗi bốn khối tích chập–gộp giống nhau, kết thúc bằng gộp toàn cục và một lớp tuyến tính. Mỗi lớp tích chập 1D thực chất là phép tương quan chéo (cross-correlation) trượt một kernel độ dài $K=5$ dọc trục thời gian; với mỗi vị trí đầu ra t và mỗi kênh ra j :

$$y_j[t] = b_j + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{j,c}[k] \cdot x_c[t+k-P]$$

trong đó $P=2$ là padding (đệm 0 hai biên để giữ độ dài), C_{in} là số kênh vào. Vì padding “same”, độ dài chuỗi không đổi qua tích chập; việc giảm chiều hoàn toàn do MaxPool đảm nhiệm. MaxPool 1D với cửa sổ và stride đều bằng 5 lấy giá trị lớn nhất trên từng cụm 5 mẫu liên tiếp, giảm độ dài đúng 5 lần mỗi lớp ($2500 \rightarrow 500 \rightarrow 100 \rightarrow 20 \rightarrow 4$):

$$p_j[t] = \max_{0 \leq k < 5} y_j[5t + k]$$

Sau bốn khối, mỗi kênh còn 4 mẫu. Global Average Pooling (GAP) nén mỗi kênh thành một số bằng trung bình trên trục thời gian, tạo vector đặc trưng độ dài bằng số kênh (8):

$$g_j = \frac{1}{4} \sum_{t=0}^3 p_j[t]$$

Vector đặc trưng đi qua lớp Fully-Connected ($8 \rightarrow 4$) cho 4 logit, rồi argmax chọn lớp có logit lớn nhất làm nhãn dự đoán:

$$z_i = \sum_j W_{i,j} g_j + b_i, \hat{y} = \arg \max_i z_i$$

- ReLU chỉ đặt trước FC (sau Conv4) là một lựa chọn có chủ đích: nhịp tim mang thông tin ở cả phần âm lẫn dương của tín hiệu, nên Conv1–3 không ReLU để không cắt mất biên độ âm; ReLU ở Conv4 tạo phi tuyến tính cuối cùng cho bộ phân loại tuyến tính phía sau.

3.3. Hàm mất mát, lan truyền ngược và bộ tối ưu

Mô hình được huấn luyện theo học có giám sát. Đầu ra 4 logit được chuẩn hóa thành phân phối xác suất bằng softmax, và hàm mất mát là cross-entropy giữa phân phối dự đoán và nhãn thật (one-hot). Với một mẫu có nhãn đúng là lớp c :

$$\mathcal{L} = -\log\left(\frac{e^{z_c}}{\sum_i e^{z_i}}\right) = -z_c + \log \sum_i e^{z_i}$$

Trong PyTorch, hai bước softmax và cross-entropy được gộp trong `nn.CrossEntropyLoss` (nhận thẳng logit, ổn định số học). Mất mát của cả batch là trung bình theo mẫu.

Quá trình lan truyền ngược (backpropagation) tính gradient của hàm mất mát theo mọi tham số bằng quy tắc chuỗi, lan ngược từ lớp FC qua GAP, các khối Conv–Pool về tới Conv1. Một bước huấn luyện gồm bốn thao tác lặp lại trên từng batch:

```
for each batch (ecg, label):
    optimizer.zero_grad()      # xóa gradient tích lũy
    logits = model(ecg)         # lan truyền tiến (forward)
    loss    = CrossEntropy(logits, label)
```

<code>loss.backward()</code>	# lan truyền ngược: tính $dL/d\theta$
<code>optimizer.step()</code>	# cập nhật tham số θ

Bộ tối ưu là Adam (Adaptive Moment Estimation) — kết hợp động lượng (moment bậc nhất) và tỉ lệ học thích nghi theo từng tham số (moment bậc hai), hội tụ nhanh và ít nhạy với việc chọn tỉ lệ học hơn SGD thuần. Với mỗi tham số θ , Adam duy trì hai ước lượng trung bình trượt của gradient g và bình phương gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Cấu hình huấn luyện thực tế:

- Hàm mất mát: CrossEntropyLoss; bộ tối ưu: Adam, tỉ lệ học khởi đầu $\eta = 1e-3$.
- Lịch tỉ lệ học: MultiStepLR giảm $\times 0,1$ tại giữa quá trình (epoch 50) $\rightarrow$ tinh chỉnh ở $1e-4$.
- 100 epoch, batch size 128; chọn checkpoint tốt nhất theo val loss thấp nhất (early-stopping ngầm theo validation).
- Cùng công thức tối ưu (Adam + CrossEntropy) được tái sử dụng ở cả hai giai đoạn sau: fine-tune sau pruning và huấn luyện QAT — chỉ khác tỉ lệ học và số epoch.

3.4. Các thuật toán tối ưu mô hình (nén & lượng tử hóa)

Ba kỹ thuật tối ưu được áp dụng tuần tự để đưa mô hình từ một CNN float32 đầy đủ về một lõi INT8 cực nhỏ, phù hợp phần cứng mà giữ độ chính xác.

(a) Cắt tỉa kênh có cấu trúc (structured channel pruning).

Mô hình gốc (kênh 4-8-8-16) được cắt tỉa xuống (4-4-8-8) bằng cách loại bỏ trọn các bộ lọc (output channel) ít quan trọng nhất — cắt tỉa “có cấu trúc” nên kết quả vẫn là một CNN dày đặc, chạy thẳng trên phần cứng mà không cần hỗ trợ ma trận thưa. Độ quan trọng của mỗi kênh được xếp hạng theo hai tiêu chí:

- Chuẩn L1 của bộ lọc: kênh có tổng trị tuyệt đối trọng số nhỏ $\rightarrow$ đóng góp ít $\rightarrow$ cắt trước.

$$I_j^{L1} = \sum_{c,k} |w_{j,c}[k]|$$

- Điểm Taylor bậc nhất: ước lượng mức tăng hàm mất mát nếu xóa kênh j bằng tích của activation và gradient, lấy trung bình trên dữ liệu — bám sát ảnh hưởng thật tới độ chính xác hơn chuẩn L1 thuần.

$$I_j^{Taylor} = \left| \frac{1}{N} \sum_n a_j^{(n)} \cdot \frac{\partial \mathcal{L}}{\partial a_j^{(n)}} \right|$$

Sau khi cắt, mô hình được tinh chỉnh lại (fine-tune) hai pha để phục hồi độ chính xác: pha 1 (30 epoch, lr=1e-3) khôi phục nhanh, pha 2 (20 epoch, lr=1e-4) tinh chỉnh. Kết quả: tham số giảm từ ~1244 xuống 654 mà accuracy gần như không đổi (~94%).

(b) Lượng tử hóa nhận biết huấn luyện (QAT) với STE và scale EMA.

Thay vì chỉ lượng tử hóa sau huấn luyện, mô hình được huấn luyện trực tiếp dưới điều kiện lượng tử hóa “giả” (fake quantization): trong forward, mỗi tensor trọng số/activation bị lượng tử hóa rồi giải-lượng-tử ngay (quantize–dequantize) để mạng “nhìn thấy” sai số lượng tử ngay khi học:

$$x_q = \text{round}\left(\frac{x}{s}\right) \cdot s, \text{clamp về } [-127, 127]$$

- Straight-Through Estimator (STE): phép round không khả vi, nên ở backward gradient được truyền thẳng qua (đạo hàm xấp xỉ bằng 1), cho phép huấn luyện qua khâu lượng tử hóa.

$$\text{forward: } x_q; \text{ backward: } \frac{\partial x_q}{\partial x} \approx 1$$

- Scale học bằng trung bình trượt mũ (EMA) của biên độ lớn nhất qua các batch, ổn định hơn lấy max tức thời; sau giai đoạn warmup, scale được “đóng băng” để hội tụ ổn định.

(c) Lượng tử hóa power-of-two (chi tiết công thức ở mục 3.5).

Khác biệt then chốt so với QAT thông thường: scale của mỗi lớp bị ràng buộc là lũy thừa của hai, biến phép rescale đắt đỏ (nhân với số thực + dịch) thành chỉ một phép dịch bit + cộng bù làm tròn. Đây chính là cầu nối giữa tối ưu phần mềm và tối ưu phần cứng: lựa chọn ở khâu huấn luyện trực tiếp xóa bỏ multiplier ở khâu rescale trên FPGA.

3.5. Lượng tử hóa power-of-two QAT — công thức

Trọng số, activation và bias được lượng tử hóa INT8 với scale là lũy thừa hai theo từng lớp. Số bit dịch của mỗi lớp được chọn theo:

Giá trị cụ thể cho từng lớp (Conv1..Conv4, FC):

$$nb = \{8, 6, 6, 7, 0\}, w_shift = \{6, 6, 6, 7, 8\}, input_shift = 2$$

Phép rescale ở đầu ra mỗi lớp dùng làm tròn nửa lên (round-half-up) rồi bão hòa về dải INT8:

$$out = \text{clamp}\left(\left\lfloor \frac{acc + 2^{nb-1}}{2^{nb}} \right\rfloor, -127, 127\right)$$

Tương đương với phép dịch bit số học có cộng bù làm tròn:

$$\text{round_half_up}(x) = (x + 2^{nb-1}) \gg nb$$

Bias được scale theo cùng hệ số và lưu INT32 little-endian:

$$\text{bias_scaled} = \text{round}(b_{float} \cdot 2^{nb})$$

Vì mọi scale là lũy thừa hai, khâu rescale chỉ cần dịch bit + cộng, tức 0 DSP — trong khi lượng tử general-scale cần 1 multiplier cho mỗi điểm rescale. Đây là đòn bẩy tiết kiệm tài nguyên và năng lượng then chốt của thiết kế.

3.6. Kết quả lượng tử

Để định lượng đánh đổi của power-of-two so với general-scale, dự án thực hiện một ma trận ablation (single-run, seed=42; cột DSP là số multiplier cần cho riêng khâu rescale):

Bảng 2. Ablation lượng tử hóa trên Chapman.

Variant	Scale	Train	Acc %	F1	DSP rescale
Float32 baseline	—	—	94,65	0,9402	—
PTQ power-of-2	2^{nb}	none	94,08	0,9338	0
PTQ general	absmax/127	none	94,46	0,9380	4
QAT power-of-2 (ours)	2^{nb}	fake-quant	94,37	0,9364	0
QAT general	absmax/127	fake-quant	94,65	0,9398	4

4. Thiết kế phần cứng dựa trên thuật toán (Hardware)

4.1. Kiến trúc tổng thể

```

Avalon-MM → Input SRAM (2500 × 8b)
              | (MUX cho Conv1)
Conv-Pool Engine (8 CP block song song,
K=5 pad=2, MaxPool /5)
              | Ping-Pong SRAM (feature map liên lớp)
GAP / FC / Argmax → result[1:0] (0..3)

```

Một FSM điều khiển phẳng 8 trạng thái tuần tự hóa 4 lớp conv cùng đuôi GAP/FC/Argmax. Module avalon_slave.v đóng vai bus adapter, dùng chung cho cả luồng synthesis (virtual-pin) lẫn luồng on-board (JTAG-to-Avalon).

4.2. CP block — pipeline 5 tầng, tách 3 submodule

- MAC (S1–S4): 5 multiplier INT8×INT8 (DSP18) → adder tree 3 tầng → tree_out.
- Accumulate + rescale (S5–S8): cộng dồn partial sum theo in_ch; gấp (fold) bias + round-add vào giá trị khởi tạo accumulator để cắt khỏi critical path (kết quả số học không đổi); clamp [-127,127]; ReLU cho Conv4.
- Pool (S9): comparator max trượt cho MaxPool K=5, stride 5.

Độ sâu pipeline từ MUX cửa số vào tới thời điểm cập nhật accumulator là đúng 5 chu kỳ; delay chain trong controller phải khớp con số này — một lỗi off-by-one ở đây từng là bug kiểm chứng chính và đã được sửa.

4.3. Dataflow streaming gấp (folded)

Thay vì map toàn bộ mỗi lớp ra phần cứng riêng (fully-mapped như competitor Liu 2023), Conv-Pool Engine time-multiplex 8 CP block trên các vị trí output và stream đầu vào qua shift-register. Đây là đòn bẩy diện tích chính: tái sử dụng một bộ PE cho mọi vị trí của một lớp, đổi độ trễ tăng (nhưng xác định) lấy mức giảm logic/register đáng kể.

4.4. Số liệu phần cứng

Thiết bị Cyclone V 5CSXFC6D6F31C6, Fmax đọc ở góc Slow 1100mV/85°C:

Bảng 3. Tài nguyên & timing trên Cyclone V.

Metric	Baseline (FF-ROM)	+ weight-RAM reload
ALM	2.201 (5%)	2.820 (7%)
DSP	28 (25%)	28 (25%)
Registers	3.177	4.852
M10K	20 (4%)	28 (5%)
Fmax (standalone)	104,85 MHz	108,94 MHz
Latency	5.216 cy (52,16 μ s)	5.216 cy (52,16 μ s)
Throughput	~19.200 inf/s	~19.200 inf/s

5. Kiểm định chức năng thiết kế (Verification)

Hợp đồng kiểm chứng của dự án là bit-exactness giữa Python và RTL. Mô hình golden Python tái lập đúng trình tự số học của RTL:

$$acc_{32} \rightarrow +bias_scaled \rightarrow +2^{nb-1} \rightarrow \gg nb \rightarrow \text{clamp}[-127,127] \rightarrow \text{ReLU}(\text{Conv4}) \\ \rightarrow \text{MaxPool}$$

Trong đó GAP dùng phép chia nguyên (floor) và FC giữ nguyên logit INT32 (nb=0) đưa vào argmax:

$$GAP = \left\lfloor \frac{sum}{4} \right\rfloor = sum \gg 2$$

Mỗi mẫu đầu vào có 21 checkpoint (input INT8, 4 pool output, GAP, 4 FC logit). Testbench Questa nạp golden .mem và so sánh từng checkpoint. Kết quả lỗi production:

- 21/21 checkpoint PASS, max|diff| = 0 LSB trên 15.312 phân tử so sánh (3 mẫu test).
- Latency xác định 5.216 chu kỳ cho mọi mẫu.

6. Kiểm chứng cross-dataset với dữ liệu Georgia

Để trả lời câu hỏi “mô hình có tổng quát hóa ra ngoài dữ liệu huấn luyện không?”, dự án đánh giá mô hình trực tiếp trên dữ liệu Georgia 12-lead (PhysioNet/CinC Challenge 2020, bộ Emory G12EC). Đây là dữ liệu thu ở một quần thể và cơ sở y tế hoàn toàn khác, đóng vai far-transfer cân bằng (balanced).

6.1. Thiết lập đánh giá

- Bộ huấn luyện mở rộng: gộp Chapman và Ningbo (cùng chuẩn 12-lead/10s/500Hz, cùng taxonomy SNOMED 4 lớp) để mở rộng độ đa dạng quần thể và thiết bị thu, tăng độ bền của boundary giữa các lớp nhịp.
- Tiền xử lý: lấy lead II từ trường val[1] của file .mat, downsample 500→250 Hz, z-score chuẩn hóa, cửa sổ 2500 mẫu — đồng nhất với pipeline Chapman.
- Ánh xạ nhãn: các SNOMED subclass của Georgia khớp đúng taxonomy 4 lớp của Chapman.
- Tập đánh giá Georgia: 5552 record, phân bố cân bằng AFIB 692 / GSVT 1242 / SB 1535 / SR 2083 — khác hẳn PTB-XL (SR chiếm 84%), nên là phép thử khắt khe về tổng quát hóa.

6.2. Kết quả huấn luyện trên Chapman mở rộng đánh giá trên Georgia

Mô hình huấn luyện trên Chapman + Ningbo (float32) được đánh giá zero-shot trực tiếp trên 5552 record Georgia. Kết quả tổng (TH1) so với baseline chỉ-Chapman làm đối chiếu:

Bảng 5. Cross-dataset trên Georgia

Chế độ	Acc	macro-P	macro-R	macro-F1	macro-AUC
Chapman/Ningbo → Georgia	0,9307	0,9123	0,9177	0,9149	0,9812

Bảng per-class (train Chapman + Ningbo → Georgia, 5552 record):

Bảng 6. Per-class (train Chapman/Ningbo) trên Georgia.

Lớp	Precision	Recall	F1	Support
AFIB	0,805	0,840	0,822	692
GSVT	0,934	0,936	0,935	1242
SB	0,957	0,956	0,956	1535
SR	0,953	0,939	0,946	2083
macro	0,912	0,918	0,915	5552

Ma trận nhầm lẫn (hàng = nhãn thật, cột = dự đoán) của TH1:

		AFIB	GSVT	SB	SR
AFIB	→	581	37	10	64
GSVT	→	58	1163	8	13
SB	→	43	6	1467	19
SR	→	40	39	48	1956

6.3. Phân tích

Train Chapman/Ningbo, test Georgia (far-transfer balanced) — cho một bức tranh tổng quát hóa đầy đủ, trải từ phân phối gần đến phân phối xa; và cho thấy chiến lược mở rộng tập huấn luyện sang dữ liệu cùng họ là hướng nâng độ bền hiệu quả mà không cần can thiệp vào dữ liệu test.

7. Demo thiết kế trên FPGA

Lỗi được nạp lên kit DE10-Standard và điều khiển qua cầu JTAG-to-Avalon kết hợp công cụ System Console của Quartus. Quy trình demo: nạp trọng số Chapman vào weight-RAM qua Avalon, ghi mẫu ECG vào input SRAM, kích hoạt inference, đọc kết quả result[1:0].

- **Kết quả:** Phân loại tập test Chapman trên board đạt 94,27% (1004/1065), tái lập kết quả mô phỏng ~94%. Chênh lệch là do test-subset/lần chạy, không phải sai số số học (datapath đã bit-exact).
- **Hạ tầng điều khiển:** JTAG-to-Avalon + System Console.

Demo này hoàn tất mục tiêu “triển khai và đo đạc thực tế trên FPGA”: lỗi không chỉ đúng trong mô phỏng mà còn cho ra cùng kết quả khi chạy trên silicon thật.

8. Báo cáo tiến độ hiện tại

Bảng 7. Trạng thái tiến độ theo Phase.

Phase	Nội dung	Trạng thái
1. Software baseline	Prune (4,4,8,8), QAT-INT8 round-half-up, export hex, golden .mem	Hoàn thành
2. Cross-dataset	Georgia (far-balanced)	Hoàn thành
3. Hardware Design	ecg_core.v + wrapper Avalon bus	Hoàn thành
4. Verification	Testbench 3 sample golden raw → int8	Hoàn thành
5. Synthesis	ALM 2201, DSP 28, Fmax 104,85	Hoàn thành
6. Demo on-board	JTAG-to-Avalon DE10 94,27%	JTAG done